

Vault-Door-3

📁 picoCTF > Reverse Engineering · + Tags

Reverse engineering explanation:

```

24     if (password.length() != 32) {
25         return false; // if password is less than 32 characters, it will return false
26     }
27     char[] buffer = new char[32]; // creates a buffer list for password
28     int i;
29     for (i=0; i<8; i++) {           // starts at 0, ends at 7
30         buffer[i] = password.charAt(i); // copies the first 8 characters of the password into the buffer (jU5t_a_s)
31     }
32     for (; i<16; i++) {           // starts at 8, ends at 15
33         buffer[i] = password.charAt(23-i); // copies the next 8 characters of the password into the buffer in reverse order (23-8) = 15,
34         // (23-15) = 8 translates to (1mpl3_an)
35     } // starts at 16, ends at 30
36     for (; i<32; i+=2) {         // starts at 16, ends at 30
37         buffer[i] = password.charAt(46-i); // copies the next 8 characters of the password into the buffer in reverse order
38         // but it fills only even numbers (46-16) = 30 = 4, (46-18) = 28 = r, (46-20) = 26 = m,
39         // (46-22) = 24 = 4, (46-24) = 22 = u, (46-26) = 20 = e, (46-28) = 18 = 0, (46-30) = 16 = c
40     } // translates to (4rm4ue0c)
41     for (i=31; i>=17; i-=2) {     // starts at 31, ends at 17
42         buffer[i] = password.charAt(i); // copies the next 8 characters of the password into the buffer but only odd numbers
43         // 31 = 2, 29 = b, 27 = 6, 25 = _, 23 = _, 21 = _, 19 = 4, 17 = g
44     } // (2b6_4g)
45     // password should be = jU5t_a_s1mpl3_an4gr4m_4_u_e60bc2
46     String s = new String(buffer);
47     return s.equals("jU5t_a_sna3lpm1cg04e_u_4_m6rb42");
48 }

```

characters added sequentially from the last 2 loops like char 16 from loop 3 then char 17 from loop 4 then again char 18 from loop 3.

the flag is: picoCTF{jU5t_a_s1mpl3_an4gr4m_4_u_e60bc2}